Tactile Classification on DE10-Lite

A Toy CNN Done by Hand

Int4 and Small Tensors

Jonathan Kirkpatrick
student id: 202230652

November 12, 2025

Introduction

Having established that our full model can perform inference with an acceptable level of fidelity, even when constrained to a limited integer domain, we turn our attention to the actual mechanics of this architectural class. We present a reduced example to highlight the underlying mathematical simplicity and to refine our intuition about the internal representations that arise as input is transformed into inference.

Even the simplest convolutional neural networks operating in the floating-point domain are difficult to analyze numerically. The sheer number of parameters and complexity of the convolution operations quickly become overwhelming, even in modest architectures. Additionally, floating-point arithmetic,while deterministic at the level of individual operations, breaks key algebraic properties such as associativity and distributivity. This leads to variation in numerical results depending on operation order, compiler behaviour, and target platform, undermining reproducibility and obscuring internal computations.

To emphasize the distinctiveness of our approach, we present a minimal CNN that operates entirely within an even tighter integer domain, while closely mirroring the model targeted for deployment on our FPGA. For this example, we restrict our parameters and input tensors to 4-bit, signed integers. We reduce the number of layers to the bare minimum necessary to retain pedagogical value. In the absence of a dataset to guide training, we substitute random numbers[A] for learned weights in this toy model and adopt the following architecture:

$$\textbf{Input} \rightarrow$$

$$\textbf{Depthwise} \rightarrow \textbf{Pointwise} \rightarrow \textbf{ReLU} \rightarrow \textbf{MaxPool} \rightarrow$$

$$\textbf{Depthwise} \rightarrow \textbf{Pointwise} \rightarrow \textbf{ReLU} \rightarrow \textbf{MaxPool} \rightarrow$$

$$\textbf{Global AvgPool} \rightarrow \textbf{Fully Connected} \rightarrow \textbf{ArgMax}$$

This configuration replicates two blocks of the pairwise separable convolutions used in our full model, followed by a global average pool, a fully connected layer, and a terminal argmax over the output logits to yield the inferred class.

1 The Mathematics of Our Model

We introduce two integer domains, they are analogous to signed int4 and signed int8 respectively:

$$\mathbb{Z}_4^{\pm} := \{x \in \mathbb{Z} | -8 \leq x \leq 7\}, \quad \mathbb{Z}_8^{\pm} := \{x \in \mathbb{Z} | -128 \leq x \leq 127\}$$

In our larger model, these are replaced with int8 and int16 respectively.

For all operations, the input tensors are bound to the $\mathbb{Z}_4^{\pm}$ domain. Operations involving multiplication (depthwise and pointwise convolutions, fully connected layer) will accumulate results in the $\mathbb{Z}_8^{\pm}$ domain to prevent overflow. We have chosen a generous accumulation width; however, one may reduce this by analyzing the worst-case accumulation for a given weight tensor, obtained by constructing a pathologically maximal input tensor. Following these accumulation steps, a right-shift operation is applied to re-quantize the values back to the $\mathbb{Z}_4^{\pm}$ domain, minimizing information loss while fitting the constrained bitwidth.

1.1 Depthwise 1D Convolutions

We define a depthwise convolution, $\boldsymbol{y} = \boldsymbol{x} \overset{\text{dw}}{*} \boldsymbol{w}$, as follows. Let $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times L}$ be a tensor consisting of C channels, each of length L, and let $\boldsymbol{w} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times k}$ be the corresponding set of depthwise convolution kernels, where each kernel has size $k < L$ and k is odd. We apply a symmetric zero-padding of width $p = \frac{k-1}{2}$, resulting in a padded input tensor $\tilde{\boldsymbol{x}} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times (L+2p)}$.

The depthwise convolution produces an output tensor $\boldsymbol{y}_{\text{acc}} \in \left(\mathbb{Z}_8^{\pm}\right)^{C \times L}$, where each channel is convolved independently. For channel $j \in \{0, \ldots, C-1\}$, the output at position $i \in \{0, \ldots, L-1\}$ is given by:

$$y_{\text{acc},i}^{(j)} = \sum_{s=0}^{k-1} w_s^{(j)} \cdot \tilde{x}_{i+s}^{(j)}$$

Following this accumulation, the values are right-shifted by a predetermined amount S_{dw} to produce the final output $\boldsymbol{y} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times L}$ for subsequent layers:

$$y_i^{(j)} = y_{\text{acc},i}^{(j)} \gg S_{dw}$$

The shift amount S_{dw} is chosen to bring the maximum possible value within range while preserving as much precision as possible. In our worked example, S_{dw} is 4 for the first depthwise convolution. In our larger model, we pass the entire dataset through to identify the greatest magnitude seen during training to determine an appropriate shift; we then clamp to $\mathbb{Z}_8^{\pm}$.

1.2 Pointwise 1D Convolutions

We define a pointwise convolution, $\boldsymbol{y} = \boldsymbol{x} \overset{\text{pw}}{*} \boldsymbol{W}$, as follows. Let $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times L}$ be a tensor as described above. A pointwise convolution is a linear transformation $\boldsymbol{W} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times C'}$, applied at each position $i \in \{0, \ldots, L-1\}$. The output $\boldsymbol{y}_{\text{acc}} \in \left(\mathbb{Z}_8^{\pm}\right)^{C' \times L}$ is computed as:

$$\boldsymbol{y}_{\text{acc}} \cdot, i := \boldsymbol{W}^\top \boldsymbol{x} \cdot, i$$

Alternatively, viewing the input as a matrix $\boldsymbol{x}^\top \in \left(\mathbb{Z}_4^{\pm}\right)^{L \times C}$, the operation becomes:

$$\boldsymbol{y}_{\text{acc}}^\top = \boldsymbol{x}^\top \boldsymbol{W}$$

It is important to note that this alternate representation holds in the dimensionality of our problem space, but does not generalize. Similar to depthwise convolutions, the accumulated results are then right-shifted by S_{pw} to return to the $\mathbb{Z}_4^{\pm}$ domain:

$$\boldsymbol{y} = \boldsymbol{y}_{\text{acc}} \gg S_{pw}$$

1.3 ReLU Activation

We define the Rectified Linear Unit (ReLU) activation function, $\boldsymbol{y} = \text{ReLU}(\boldsymbol{x})$, applied element-wise to an input tensor $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C \times L}$. For each element $x_{j,i}$ in the input tensor, the corresponding output $y_{j,i}$ is given by:

$$y_{j,i} = \max(0, x_{j,i})$$

The ReLU operation is non-linear and introduces sparsity by clamping all negative values to zero. Since the operation only modifies the sign of negative values or leaves positive values unchanged, the output tensor $\boldsymbol{y}$ remains within the $\mathbb{Z}_4^{\pm}$ domain.

1.4 Max Pooling

We define a max pooling operation, $\boldsymbol{y} = \boldsymbol{x}\overset{\mathrm{k}}{\vee}$, as follows. Let $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C\times L}$ be a tensor. For each channel $j \in \{0, \ldots, C-1\}$, the operation downsamples the feature map by selecting the maximum value within non-overlapping windows of size k. The stride of the pooling operation is equal to the kernel size k and $k|L$.

The output tensor $\boldsymbol{y} \in \left(\mathbb{Z}_4^{\pm}\right)^{C\times\left(\frac{L}{k}\right)}$ has reduced length. For each channel j, the output at position $i \in \{0, \ldots, \left(\frac{L}{k}\right) - 1\}$ is given by:

$$y_i^{(j)} = \max_{s=0}^{k-1} x_{i\cdot k+s}^{(j)}$$

Max pooling does not introduce any new parameters and simply propagates the maximum value within a receptive field, helping to reduce the dimensionality of the feature maps and provide a degree of translational invariance. Since the input to max pooling is from $\mathbb{Z}_4^{\pm}$, the output also remains within $\mathbb{Z}_4^{\pm}$.

1.5 Global Average Pooling

We define a global average pooling operation, $\boldsymbol{y} = \mathrm{GlobalAvgPool}(\boldsymbol{x})$, applied to an input tensor $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C\times L}$. For each channel $j \in \{0, \ldots, C-1\}$, the operation computes the average of all values within that channel, reducing the length of each channel to a single value. The output tensor $\boldsymbol{y} \in \left(\mathbb{Z}_4^{\pm}\right)^{C\times 1}$ (or simply $\mathbb{Z}_4^{\pm C}$) is computed as:

$$y^{(j)} = \sum_{i=0}^{L-1} x_i^{(j)} \gg \log_2 L$$

In our integer-only implementation, this average is performed via summation followed by a right-shift to approximate the division. We have ensured our final tensor has a length equal to some power of 2. This produces an approximation or a specific scaling strategy for the average. The output of global average pooling is also kept within the $\mathbb{Z}_4^{\pm}$ domain.

1.6 Fully Connected Layer

We define a fully connected layer, $\boldsymbol{y} = \mathrm{FC}(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{b})$. This operation takes a flattened input tensor $\boldsymbol{x} \in \left(\mathbb{Z}_4^{\pm}\right)^{C_{\mathrm{in}}\times 1}$ and transforms it into an output tensor $\boldsymbol{y} \in \left(\mathbb{Z}_8^{\pm}\right)^{C_{\mathrm{out}}\times 1}$ using a weight matrix $\boldsymbol{W} \in \left(\mathbb{Z}_4^{\pm}\right)^{C_{\mathrm{out}}\times C_{\mathrm{in}}}$ and a bias vector $\boldsymbol{b} \in \left(\mathbb{Z}_4^{\pm}\right)^{C_{\mathrm{out}}\times 1}$. The operation is defined as:

$$\boldsymbol{y}_{\mathrm{acc}} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

The matrix multiplication $\boldsymbol{W}\boldsymbol{x}$ and the subsequent addition of $\boldsymbol{b}$ accumulate results in the $\mathbb{Z}_8^{\pm}$ domain to avoid overflow. Following this, a final right-shift S_{fc} may be applied if a subsequent layer expects $\mathbb{Z}_4^{\pm}$ input. However, since this is the final layer before ArgMax, the logits can remain in $\mathbb{Z}_8^{\pm}$.

1.7 ArgMax

The ArgMax operation, $\mathrm{ArgMax}(\boldsymbol{y})$, is the final step in the classification process. It takes the output logits $\boldsymbol{y} \in \left(\mathbb{Z}_8^{\pm}\right)^{C_{\mathrm{out}}\times 1}$ from the fully connected layer and determines the index of the maximum value.

This index corresponds to the inferred class.

$$\text{Class} = \arg\max_k(y_k)$$

This operation does not involve any arithmetic and simply identifies the highest confidence score among the output classes.

Summary of Functions

Layer Type	Operator	Symbolic Representation
Depthwise 1D Convolution	$\boldsymbol{y} = \boldsymbol{x} \overset{\text{dw}}{*} \boldsymbol{w}$	$y_i^{(j)} = \left(\sum_{s=0}^{k-1} w_s^{(j)} \cdot \tilde{x}_{i+s}^{(j)}\right) \gg S_{dw}$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C\times L},\ \boldsymbol{w} \in (\mathbb{Z}_4^{\pm})^{C\times k}$ $j \in \{0,\ldots,C{-}1\},\ i \in \{0,\ldots,L{-}1\}$
Pointwise 1D Convolution	$\boldsymbol{y} = \boldsymbol{x} \overset{\text{pw}}{*} \boldsymbol{W}$	$\boldsymbol{y}_{\cdot,i} = \left(\boldsymbol{W}^\top \boldsymbol{x}_{\cdot,i}\right) \gg S_{pw}$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C\times L},\ \boldsymbol{W} \in (\mathbb{Z}_4^{\pm})^{C\times C'}$ $i \in \{0,\ldots,L{-}1\}$
ReLU Activation	$\boldsymbol{y} = \text{ReLU}(\boldsymbol{x})$	$y_i^{(j)} = \max(0, x_i^{(j)})$ $\boldsymbol{x}, \boldsymbol{y} \in (\mathbb{Z}_4^{\pm})^{C\times L}$
Max Pooling	$\boldsymbol{y} = \boldsymbol{x} \overset{\text{k}}{\vee}$	$y_i^{(j)} = \max\{x_i, x_{i+1}, \ldots, x_{i+k-1}\}^{(j)}$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C\times L},\ \boldsymbol{y} \in (\mathbb{Z}_4^{\pm})^{C\times(L-k+1)}$
Global Average Pooling	$\boldsymbol{y} = \text{GAP}(\boldsymbol{x})$	$y^{(j)} = \left(\sum_{i=0}^{L-1} x_i^{(j)}\right) \gg \log_2 L$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C\times L},\ \boldsymbol{y} \in (\mathbb{Z}_4^{\pm})^{C}$
Fully Connected Layer	$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$	$\boldsymbol{y}_{\text{acc}} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \quad \boldsymbol{y} = \boldsymbol{y}_{\text{acc}} \gg S_{fc}$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C},\ \boldsymbol{W} \in (\mathbb{Z}_4^{\pm})^{C\times C'},\ \boldsymbol{b} \in (\mathbb{Z}_4^{\pm})^{C'}$
ArgMax	$j = \arg\max(\boldsymbol{x})$	$j = \arg\max_i x_i$ $\boldsymbol{x} \in (\mathbb{Z}_4^{\pm})^{C}$

Tensor Evolution in Our Small Model

In figure 1, we track the progression of the intermediate tensor shape after each of the operations.

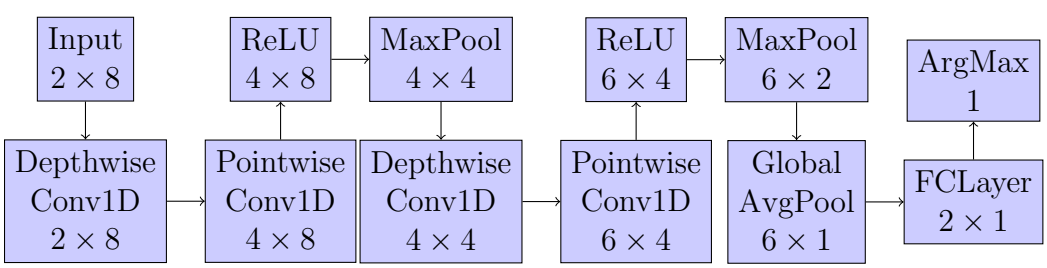

Figure 1: Tensor Evolution

2 Inference by Hand

Step 1: Randomly generate all necessary weights^A from a `kaiming uniform`^{B} distribution.

Step 2: Scale and round the weights to saturate $\mathbb{Z}_4^{\pm}$.

Step 3: Generate a random input tensor $\boldsymbol{x}$ from a uniform distribution over $\left(\mathbb{Z}_4^{\pm}\right)^{2\times 8}$.

$$\boldsymbol{x} = \begin{bmatrix} -2 & -5 & 4 & 6 & 2 & -1 & 4 & -1 \\ -2 & 1 & -6 & -2 & 2 & 2 & -1 & -4 \end{bmatrix}$$

Step 4: Perform the first Depthwise 1D Convolution $\boldsymbol{y} = \boldsymbol{x} \overset{\text{dw}}{*} \boldsymbol{w_1}$ using a kernel size $k = 5$ and shift factor $S_{dw} = 4$.

$$\begin{aligned}
\boldsymbol{y_1} &= \begin{bmatrix} -2 & -5 & 4 & 6 & 2 & -1 & 4 & -1 \\ -2 & 1 & -6 & -2 & 2 & 2 & -1 & -4 \end{bmatrix} \overset{\text{dw}}{*} \begin{bmatrix} -2 & 7 & 3 & 1 & -6 \\ -6 & -7 & 5 & 1 & 3 \end{bmatrix} \\
&= \begin{bmatrix} -35 & -61 & -25 & 64 & 15 & 9 & 0 & 27 \\ -27 & 7 & -21 & 34 & 59 & -5 & -35 & -25 \end{bmatrix} \gg 4 \\
&= \begin{bmatrix} -3 & -4 & -2 & 4 & 0 & 0 & 0 & 1 \\ -2 & 0 & -2 & 2 & 3 & -1 & -3 & -2 \end{bmatrix}
\end{aligned}$$

Step 5: Perform the first Pointwise 1D Convolution $\boldsymbol{y_2} = \boldsymbol{y_1} \overset{\text{pw}}{*} \boldsymbol{W_2}$ with $C = 2$ input channels, $C' = 4$ output channels and $S_{pw} = 2$.

$$\begin{aligned}
\boldsymbol{y_2} &= \left(\begin{bmatrix} -3 & -2 \\ -4 & 0 \\ -2 & -2 \\ 4 & 2 \\ 0 & 3 \\ 0 & -1 \\ 0 & -3 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} -7 & 4 & -5 & -3 \\ 6 & -5 & -5 & 0 \end{bmatrix} \right)^{\top} \gg 2 \\
&= \begin{bmatrix} 9 & 28 & 2 & -16 & 18 & -6 & -18 & -19 \\ -2 & -16 & 2 & 6 & -15 & 5 & 15 & 14 \\ 25 & 20 & 20 & -30 & -15 & 5 & 15 & 5 \\ 9 & 12 & 6 & -12 & 0 & 0 & 0 & -3 \end{bmatrix} \gg 2
\end{aligned}$$

$$\boldsymbol{y_2} = \begin{bmatrix} 2 & 7 & 0 & -4 & 4 & -2 & -5 & -5 \\ -1 & -4 & 0 & 1 & -4 & 1 & 3 & 3 \\ 6 & 5 & 5 & -8 & -4 & 1 & 3 & 1 \\ 2 & 3 & 1 & -3 & 0 & 0 & 0 & -1 \end{bmatrix}$$

Step 6: Perform our first ReLU activation $\boldsymbol{y_3} = \text{ReLU}(\boldsymbol{y_2})$, replacing negative values with 0.

$$\boldsymbol{y_3} = \begin{bmatrix} 2 & 7 & 0 & 0 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 3 & 3 \\ 6 & 5 & 5 & 0 & 0 & 1 & 3 & 1 \\ 2 & 3 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Step 7: We reach our first MaxPool layer $\boldsymbol{y_4} = \boldsymbol{y_3}\overset{2}{\vee}$. Here we have kernel = stride = 2.

$$\boldsymbol{y_4} = \begin{bmatrix} 7 & 0 & 4 & 0 \\ 0 & 1 & 1 & 3 \\ 6 & 5 & 1 & 3 \\ 3 & 1 & 0 & 0 \end{bmatrix}$$

Step 8: Our second Depthwise 1D Convolution $\boldsymbol{y_5} = \boldsymbol{y_4} \overset{\text{dw}}{*} \boldsymbol{w_3}$ with $k = 3$ and $S_{dw} = 2$.

$$\begin{aligned} \boldsymbol{y_5} &= \begin{bmatrix} 7 & 0 & 4 & 0 \\ 0 & 1 & 1 & 3 \\ 6 & 5 & 1 & 3 \\ 3 & 1 & 0 & 0 \end{bmatrix} \overset{\text{dw}}{*} \begin{bmatrix} -2 & -4 & 1 \\ -6 & -4 & -3 \\ -1 & 4 & -5 \\ 0 & 1 & -7 \end{bmatrix} \\ &= \begin{bmatrix} -28 & -10 & -16 & -8 \\ -3 & -7 & -19 & -18 \\ -1 & 9 & -16 & 11 \\ -4 & 1 & 0 & 0 \end{bmatrix} \gg 2 \\ &= \begin{bmatrix} -7 & -3 & -4 & -2 \\ -1 & -2 & -5 & -5 \\ -1 & 2 & -4 & 2 \\ -1 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

Step 9: The second Pointwise 1D Convolution $\boldsymbol{y_6} = \boldsymbol{y_5} \overset{\text{pw}}{*} \boldsymbol{W_4}$ takes us from $C = 4$ channels to $C' = 6$ with $S_{pw} = 4$.

$$\boldsymbol{y_6} = \left(\begin{bmatrix} -7 & -1 & -1 & -1 \\ -3 & -2 & 2 & 0 \\ -4 & -5 & -4 & 0 \\ -2 & -5 & 2 & 0 \end{bmatrix} \begin{bmatrix} 1 & 6 & 2 & -7 & -3 & 7 \\ -5 & 4 & -1 & 6 & 0 & 4 \\ -7 & -3 & -6 & -4 & 0 & 6 \\ 6 & -6 & -1 & 2 & -5 & 5 \end{bmatrix} \right)^{\top} \gg 4$$

$$= \begin{bmatrix} -1 & -7 & 49 & 9 \\ -37 & -32 & -32 & -38 \\ -6 & -16 & 21 & -11 \\ 45 & 1 & 14 & -24 \\ 26 & 9 & 12 & 6 \\ -64 & -17 & -72 & -22 \end{bmatrix} \gg 4$$

$$= \begin{bmatrix} -1 & -1 & 3 & 0 \\ -3 & -2 & -2 & -3 \\ -1 & -1 & 1 & -1 \\ 2 & 0 & 0 & -2 \\ 1 & 0 & 0 & 0 \\ -4 & -2 & -5 & -2 \end{bmatrix}$$

Step 10: Once again, we perform a ReLU activation $\boldsymbol{y_7} = \text{ReLU}(\boldsymbol{y_6})$.

$$\boldsymbol{y_7} = \begin{bmatrix} 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Step 11: This second MaxPool operation $\boldsymbol{y_8} = \boldsymbol{y_7} \overset{2}{\vee}$ reduces the temporal dimension from 4 to 2.

$$\boldsymbol{y_8} = \begin{bmatrix} 0 & 3 \\ 0 & 0 \\ 0 & 1 \\ 2 & 0 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Step 12: The Global Average Pool $\boldsymbol{y_9} = \text{GAP}(\boldsymbol{y_8})$ reduces the temporal dimension to 1 by summing the two remaining values and right shifting by 1.

$$\boldsymbol{y_9} = \begin{bmatrix} 3 \\ 0 \\ 1 \\ 2 \\ 1 \\ 0 \end{bmatrix} \gg 1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Step 13: In the Fully Connected Layer $\boldsymbol{y_{10}} = \boldsymbol{W_5}\boldsymbol{y_9} + \boldsymbol{b}$ we reduce our $C = 6$ down to the final $C' = 2$ and apply our bias.

$$\boldsymbol{y_{10}} = \begin{bmatrix} 1 & 6 & -7 & -5 & -7 & -3 \\ -2 & -4 & 5 & -3 & -4 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -7 \\ 5 \end{bmatrix} = \begin{bmatrix} -11 \\ 0 \end{bmatrix}$$

Step 14: The final operation, ArgMax simply identifies the largest value among the logits in $\boldsymbol{y_{10}}$. We see that $-11 < 0$, so we may conclude that the inferred class is 1.

Conclusion

This exercise has walked through a complete, hand-constructed inference pass of a miniature convolutional neural network, constrained entirely to the $\mathbb{Z}_4^{\pm}$ and $\mathbb{Z}_8^{\pm}$ domains. Each operation, from convolution to pooling to activation, was performed explicitly, with quantized weights and integer arithmetic, in a pipeline mirroring the model architecture targeted for our FPGA deployment.

We have reinforced several key ideas:

- Quantized models, though numerically limited, can preserve the essential structure and behaviour of their full-precision counterparts, provided careful attention is paid to accumulation width and rescaling.
- Integer arithmetic need not obscure model internals; on the contrary, it can offer enhanced transparency and determinism, especially in resource-constrained environments.
- Even a simple model can exhibit the full complexity of a typical inference pipeline, and working through such an example by hand sharpens our intuition for how data flows and transforms through the network.

By tracing this path step-by-step, we hope to have demystified some of the inner workings of integer-only inference and demonstrated that, with a carefully crafted architecture and quantization-aware design, meaningful classification can emerge from surprisingly small and interpretable models.

A Randomly Generated Tensors

- The Input Tensor

$$\begin{bmatrix} -2 & -5 & 4 & 6 & 2 & -1 & 4 & -1 \\ -2 & 1 & -6 & -2 & 2 & 2 & -1 & -4 \end{bmatrix}$$

- First Depthwise Convolution Weights

$$\begin{bmatrix} -0.1122 & 0.4031 & 0.2075 & 0.0882 & -0.3077 \\ -0.3077 & -0.3953 & 0.3275 & 0.0904 & 0.1861 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 17.3641x \rfloor} \begin{bmatrix} -2 & 7 & 3 & 1 & -6 \\ -6 & -7 & 5 & 1 & 3 \end{bmatrix}$$

- First Pointwise Convolution Weights

$$\begin{bmatrix} -0.678 & 0.6646 \\ 0.4701 & -0.4068 \\ -0.45 & -0.4477 \\ -0.2768 & 0.035 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 10.3245x \rfloor} \begin{bmatrix} -7 & 6 \\ 4 & -5 \\ -5 & -5 \\ -3 & 0 \end{bmatrix}$$

- Second Depthwise Convolution Weights

$$\begin{bmatrix} -0.0786 & -0.2411 & 0.1292 \\ -0.4163 & -0.24 & -0.1543 \\ -0.0507 & 0.3293 & -0.3468 \\ 0.0164 & 0.1067 & -0.5237 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 13.3661x \rfloor} \begin{bmatrix} -2 & -4 & 1 \\ -6 & -4 & -3 \\ -1 & 4 & -5 \\ 0 & 1 & -7 \end{bmatrix}$$

- Second Pointwise Convolution Weights

$$\begin{bmatrix} 0.1075 & -0.3295 & -0.4349 & 0.4489 \\ 0.4656 & 0.3084 & -0.1954 & -0.4023 \\ 0.1842 & -0.0598 & -0.378 & -0.0048 \\ -0.4656 & 0.4093 & -0.2412 & 0.1625 \\ -0.1883 & 0.0201 & 0.0467 & -0.3151 \\ 0.4696 & 0.2751 & 0.4395 & 0.3948 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 14.9068x \rfloor} \begin{bmatrix} 1 & -5 & -7 & 6 \\ 6 & 4 & -3 & -6 \\ 2 & -1 & -6 & -1 \\ -7 & 6 & -4 & 2 \\ -3 & 0 & 0 & -5 \\ 7 & 4 & 6 & 5 \end{bmatrix}$$

- Fully Connected Layer Weights

$$\begin{bmatrix} 0.0799 & 0.3445 & -0.336 & -0.2482 & -0.3713 & -0.1426 \\ -0.0909 & -0.1867 & 0.2684 & -0.117 & -0.1789 & 0.0349 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 18.8516x \rfloor} \begin{bmatrix} 1 & 6 & -7 & -5 & -7 & -3 \\ -2 & -4 & 5 & -3 & -4 & 0 \end{bmatrix}$$

- Fully Connected Bias

$$\begin{bmatrix} -0.2932 \\ 0.2467 \end{bmatrix} \xrightarrow[\mathbb{Z}_4^{\pm}]{\lfloor 23.8758x \rfloor} \begin{bmatrix} -7 \\ 5 \end{bmatrix}$$

B Kaiming Uniform Distribution

In their 2015 paper *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, He et al. propose a successful method of initializing parameter weights in a ReLU activated CNN. Previous methods of initialization are shown to prohibit rapid convergence of sufficiently deep models, if they ever converge at all. Their distribution aims to limit the growth of intermediate activations at the start of training, by considering the breadth of the receptive field and sampling from a distribution more suited to such activations. Previously, sigmoid or tanh were widely used and helped to cap the potential intermediate value explosion by capping their output at one. The ReLU activation function, by contrast, places no such hard upper bound. Below is an approximate rendering of this distribution we used to sample our random weights above.

$$\text{weight} \in \mathcal{U}\left(-\sqrt{\frac{1}{\text{fan in}}}, \sqrt{\frac{1}{\text{fan in}}}\right), \quad \text{bias} \in \mathcal{U}\left(-\frac{1}{\sqrt{\text{fan in}}}, \frac{1}{\sqrt{\text{fan in}}}\right)$$

Where "fan in" is the number of input units.

In our present exploration, this attention to prequantized weight distribution is undone by our subsequent saturation of $\mathbb{Z}_4^{\pm}$, however it provides a shared starting point for future work related to gradient surrogates in integer model training.